



RAPPORT DE PROJET — MODULE NOSQL

GestiStock

Système de gestion de stock en temps réel adossé à Redis (Upstash)

Rapport technique présentant l'architecture, la modélisation des données et le fonctionnement complet d'une application web développée avec **Streamlit** et une base de données clé-valeur **Redis**, dans le cadre de l'examen du 4^e semestre du module Bases de données NoSQL.

Python 3.10+

Streamlit

Redis / Upstash

Architecture Key-Value

Déploiement Cloud

RÉALISÉ PAR

KEMA Didier • Mouanga Gloire • MATSIONA Aude

Présenté par le Groupe 3

Présenté par

M. OBANDA Jefferson

Juillet 2026

Table des matières

01	Introduction	
02	Présentation générale du projet	
	– Contexte académique	
	– Objectifs du projet	
	– Périmètre fonctionnel	
03	Cas d'usage	
	– Acteur et scénario métier	
	– Cas d'usage détaillés (UC1 à UC5)	
04	Choix technologiques	
	– Pourquoi une base NoSQL	
	– Pourquoi Redis (clé-valeur)	
	– Pourquoi Streamlit	
05	Choix de modélisation des données	
	– Principes de modélisation Redis	
	– Justification structure par structure	
06	Schéma de données	
	– Vue d'ensemble des clés Redis	
	– Détail des structures	
	– Cycle de vie d'un produit	
07	Architecture logicielle	
	– Architecture en couches	
	– Organisation des fichiers	
	– Flux d'une requête	
08	Fonctionnement détaillé de l'application	
	– Dashboard	
	– Produits	
	– Mouvements	
	– Historique	
	– Alertes	
09	Règles métier et gestion des erreurs	
10	Sécurité et configuration	

11	Installation et déploiement	
12	Stack technique	
13	Limites et perspectives d'évolution	
14	Conclusion	

Introduction

Ce document constitue le rapport technique du projet **GestiStock**, réalisé dans le cadre de l'examen du module **Bases de données NoSQL** (4^e semestre, 3^e année Data Science, ECES). Il a pour objectif de présenter de manière complète le fonctionnement de l'application développée : le problème métier traité, les choix technologiques et de modélisation opérés, le schéma de données retenu, ainsi que l'architecture logicielle mise en place pour y répondre.

L'exercice imposé par le cahier des charges consistait à concevoir une application exploitant une base de données **clé-valeur (Redis)** pour résoudre un cas d'usage réel, en tirant parti des structures de données natives offertes par ce type de système plutôt que de les contourner par une modélisation relationnelle déguisée. Le choix s'est porté sur la gestion de stock en temps réel, un domaine où les propriétés intrinsèques de Redis — opérations atomiques, ensembles triés, listes bornées, expiration automatique — trouvent une application directe et pédagogiquement pertinente.

L'application est accessible en ligne à l'adresse <https://didier-gloire-aude.streamlit.app/> et son code source est hébergé sur le dépôt GitHub github.com/juniorkema364/streamlit_project.

Lecture du rapport. Les chapitres 4 et 5 (choix technologiques et de modélisation) et le chapitre 6 (schéma de données) constituent le cœur de la réflexion NoSQL du projet. Les chapitres 7 et 8 détaillent ensuite l'implémentation concrète de cette modélisation dans l'application.

Présentation générale du projet

2.1 — Contexte académique

Le projet s'inscrit dans le cadre du module **Bases de données NoSQL**, dont le cahier des charges impose la réalisation d'une application fonctionnelle utilisant exclusivement un système de gestion de base de données NoSQL de type **clé-valeur** — en l'occurrence **Redis**, hébergé sur la plateforme cloud gratuite **Upstash**. L'application devait être déployée publiquement (Streamlit Community Cloud) et son code source publié sur un dépôt GitHub public.

2.2 — Objectif du projet

GestiStock est un tableau de bord web permettant à une petite chaîne de magasins de piloter son stock en temps réel : suivre les quantités disponibles pour chaque produit, enregistrer les réceptions fournisseurs et les ventes, consulter l'historique des mouvements, et être alertée automatiquement dès qu'un produit passe sous son seuil critique de réapprovisionnement.

2.3 — Périmètre fonctionnel

Référentiel produits

Création, modification, suppression et recherche de produits (SKU, nom, catégorie, prix, seuil, fournisseur).

Mouvements de stock

Enregistrement atomique des entrées (réceptions) et sorties (ventes), avec interdiction du stock négatif.

Historique

Traçabilité des 50 derniers mouvements par produit, consultation des 20 plus récents.

Alertes automatiques

Détection et classement des produits sous seuil critique, du plus urgent au moins urgent.

Statistiques en temps réel

KPI globaux : nombre de produits, alertes actives, entrées/sorties glissantes sur 24 heures.

Déploiement cloud

Base Redis hébergée sur Upstash, application déployée sur Streamlit Community Cloud.

Cas d'usage

3.1 — Acteur et scénario métier

L'unique acteur de l'application est le **gestionnaire de magasin** (ou magasinier), qui interagit avec l'ensemble des fonctionnalités via l'interface Streamlit. Aucune distinction de rôle ou d'authentification n'est mise en œuvre : il s'agit d'un outil de gestion interne, à usage unique, conforme au périmètre pédagogique du projet.

3.2 — Cas d'usage détaillés

UC1 — Créer et administrer le référentiel produits

Le gestionnaire crée une fiche produit (SKU, nom, catégorie, prix unitaire, seuil d'alerte, fournisseur, stock initial). Il peut ensuite la modifier partiellement (par exemple ajuster un prix ou un seuil), la rechercher par SKU ou par nom, ou la supprimer définitivement — la suppression entraînant la purge de toutes les données associées (stock, historique, alerte).

UC2 — Réceptionner une livraison fournisseur (entrée de stock)

À la réception d'une commande, le gestionnaire sélectionne le produit concerné, saisit la quantité reçue et un commentaire libre (numéro de bon de livraison, date). Le stock est incrémenté de façon atomique et le mouvement est journalisé.

UC3 — Enregistrer une vente (sortie de stock)

Lors d'une vente, le gestionnaire sélectionne le produit et la quantité vendue. Le système vérifie que le stock disponible est suffisant avant de décrémenter : toute tentative de sortie supérieure au stock disponible est rejetée avec un message explicite, garantissant qu'un stock ne devient jamais négatif.

UC4 — Consulter l'historique d'un produit

Le gestionnaire sélectionne un produit et visualise ses mouvements les plus récents (type, quantité, horodatage, commentaire), sous forme de journal chronologique ou de tableau exportable.

UC5 — Surveiller les alertes de rupture

Le gestionnaire consulte à tout moment la liste des produits passés sous leur seuil critique, triée du plus urgent au moins urgent, avec un niveau de sévérité indicatif (rupture totale, très critique,

stock bas). Un résumé des alertes est également visible en permanence dans la barre latérale de navigation.

UC6 — Piloter l'activité depuis le tableau de bord

Dès la connexion, le gestionnaire dispose d'une vue synthétique : nombre total de produits, nombre de produits en alerte, entrées et sorties des dernières 24 heures, top 3 des stocks les plus faibles, et vue d'ensemble complète du référentiel produits.

Choix technologiques

4.1 — Pourquoi une base de données NoSQL ?

La gestion de stock en temps réel se caractérise par un volume élevé d'opérations simples et fréquentes (incrémenter/décroître une quantité, ajouter un événement à un journal, vérifier un seuil) qui nécessitent avant tout de la **vitesse** et de l'**atomicité**, plutôt que des jointures complexes ou des transactions multi-tables. Ce profil correspond précisément aux forces d'une base clé-valeur en mémoire, et justifie l'écart avec une base relationnelle classique.

4.2 — Pourquoi Redis ?

Redis a été retenu, conformément au cahier des charges du module, pour les raisons suivantes, directement exploitées dans l'application :

- **Compteurs atomiques** (`INCRBY` / `DECRBY`) : garantissent qu'un mouvement de stock est appliqué de manière indivisible, même en cas d'accès concurrents.
- **Sorted Set (ZSET)** : permet de maintenir une liste de produits en alerte automatiquement triée par urgence, sans calcul de tri côté application.
- **List** avec `LPUSH` / `LTRIM` : offre un journal chronologique borné en une seule structure, sans purge manuelle.
- **TTL (Time To Live)** : permet des compteurs statistiques « glissants » sur 24 heures qui s'auto-expirent, sans tâche planifiée (cron) additionnelle.
- **Hébergement Upstash** : offre managée, gratuite pour un usage académique, sans carte bancaire, accessible en TLS depuis Streamlit Cloud.

4.3 — Pourquoi Streamlit ?

Streamlit permet de construire une interface web interactive entièrement en Python, sans développement front-end séparé (HTML/JS), ce qui est cohérent avec l'objectif pédagogique du module — centrer l'effort sur la modélisation et la manipulation des données NoSQL plutôt que sur l'ingénierie d'interface. Son modèle de rendu réactif (re-exécution du script à chaque interaction) s'accorde par ailleurs naturellement avec des données consultées en temps réel depuis Redis.

Bibliothèque cliente. Le client officiel `redis-py` est utilisé pour toutes les communications avec la base, en mode `decode_responses=True` afin de manipuler directement des chaînes Python plutôt que des octets bruts.

Choix de modélisation des données

5.1 — Principes directeurs

À la différence d'une base relationnelle où l'on modélise d'abord des entités et leurs relations, la modélisation Redis part des **accès applicatifs** : pour chaque opération métier fréquente (lire le stock d'un produit, trier les alertes, journaliser un mouvement), on choisit la structure de donnée native qui rend cette opération la plus rapide et la plus simple possible. Trois principes ont guidé les choix du projet :

- **Une responsabilité par clé** : chaque type d'information (fiche produit, stock, historique) vit dans sa propre clé, préfixée par son domaine, plutôt que d'être agrégée dans un objet unique volumineux.
- **La structure au service de l'algorithme** : le tri des alertes, la troncature de l'historique ou l'expiration des statistiques sont délégués aux capacités natives de Redis (ZSET, LTRIM, TTL) plutôt que recalculés côté application.
- **Aucune commande Redis hors de la couche d'accès** : toutes les interactions avec la base sont centralisées dans une unique classe `RedisManager` (fichier `database.py`), qui constitue la seule porte d'entrée vers les données. Les pages Streamlit ne connaissent jamais le détail des commandes Redis sous-jacentes.

5.2 — Justification structure par structure

HASH — Fiche produit

Un **HASH** regroupe naturellement les attributs d'une même entité (nom, catégorie, prix, seuil, fournisseur) sous une seule clé, avec la possibilité de lire ou modifier un champ isolément (`HSET` partiel) sans recharger tout l'objet — utile pour la mise à jour d'un seul champ comme le prix.

STRING — Compteur de stock

Le stock est isolé du HASH produit dans une clé **STRING** dédiée, précisément pour pouvoir exploiter `INCRBY` / `DECRBY`, des opérations atomiques que Redis ne propose pas sur un champ de HASH pris isolément dans un contexte concurrent aussi directement. Séparer « ce qui décrit le produit » de « ce qui varie à chaque vente » est également une bonne pratique de modélisation clé-valeur : les données de fréquence de mise à jour différente sont isolées.

LIST — Historique des mouvements

Chaque mouvement est sérialisé en JSON et empilé en tête de liste (`LPUSH`), puis la liste est tronquée (`LTRIM`) aux 50 éléments les plus récents. Cette approche évite la croissance illimitée d'une clé et offre nativement l'ordre chronologique inverse (du plus récent au plus ancien) recherché par l'interface.

ZSET (Sorted Set) — Alertes de rupture

Le choix le plus caractéristique du projet : les produits en alerte sont indexés dans un **unique** Sorted Set global, où le score est directement le stock actuel. Cela permet d'obtenir en une seule commande (`ZRANGE ... WITHSCORES`) la liste des produits critiques déjà triée du plus urgent au moins urgent, sans avoir à parcourir puis trier tous les produits côté application.

STRING + TTL — Statistiques glissantes 24h

Les compteurs d'entrées/sorties utilisent une clé STRING incrémentée à chaque mouvement, dont le TTL est reposé à 86 400 secondes (24h) à chaque écriture. La clé s'auto-détruit si elle n'est plus alimentée, offrant un indicateur « glissant » sans tâche de nettoyage périodique.

Schéma de données

6.1 — Vue d'ensemble des clés Redis

L'ensemble du modèle de données tient en six familles de clés, résumées ci-dessous. Le SKU (identifiant unique du produit, ex. `SKU-001`) agit comme clé de partitionnement logique entre les différentes structures relatives à un même produit.

Clé Redis	Type	Contenu / Rôle	Commandes principales
<code>produit:<sku></code>	HASH	Fiche produit : nom, catégorie, prix_unitaire, seuil_alerte, fournisseur	HSET, HGETALL, HGET
<code>stock:<sku></code>	STRING	Quantité en stock actuelle (entier)	SET, GET, INCRBY, DECRBY
<code>historique:<sku></code>	LIST	50 derniers mouvements (JSON), du plus récent au plus ancien	LPUSH, LTRIM, LRANGE
<code>alertes:stock_bas</code>	ZSET	SKU des produits en rupture, score = stock actuel	ZADD, ZREM, ZRANGE
<code>stats:entrees:24h</code>	STRING + TTL	Compteur d'entrées glissant sur 24h	INCR, EXPIRE, GET
<code>stats:sorties:24h</code>	STRING + TTL	Compteur de sorties glissant sur 24h	INCR, EXPIRE, GET

6.2 — Exemple concret de clés pour un produit

```
produit:SKU-001  HASH
  nom           = "Riz parfumé 5kg"
  categorie     = "Alimentaire"
  prix_unitaire = "6500"
  seuil_alerte  = "10"
  fournisseur   = "Grossiste Poto-Poto"

stock:SKU-001    STRING = "42"

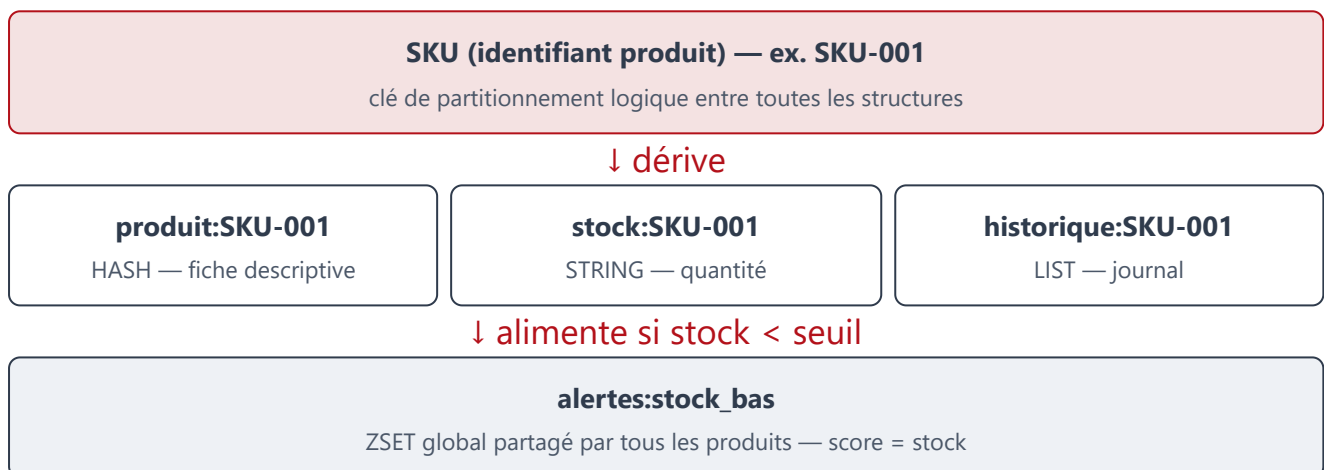
historique:SKU-001  LIST  (50 éléments max, LTRIM)
  [0] {"type":"sortie","quantite":3,"timestamp":"2026-07-13T09:12:00+00:00","commentaire":"Vente comptoir"}
```

```
[1] {"type":"entree","quantite":20,"timestamp":"2026-07-10T14:32:00+00:00","commentaire":"BL n°4521"}
...

alertes:stock_bas ZSET (présent uniquement si stock < seuil)
"SKU-045" -> score 2
"SKU-012" -> score 4
"SKU-001" -> score 8 (absent ici car 42 ≥ 10)
```

6.3 — Schéma relationnel logique des structures

Bien que Redis ne dispose pas de relations formelles, un lien logique implicite unit toutes les clés relatives à un même produit via le SKU, comme illustré ci-dessous.



6.4 — Cycle de vie d'un produit

- 1 **Création** — `add_product()` crée `produit:<sku>` (HASH) et `stock:<sku>` (STRING), puis évalue immédiatement l'alerte.
- 2 **Mouvement de stock** — `add_stock()` / `remove_stock()` appliquent `INCRBY` / `DECRBY` sur `stock:<sku>`, journalisent l'événement dans `historique:<sku>`, recalculent l'alerte et incrémentent le compteur statistique du jour.
- 3 **Mise à jour** — `update_product()` modifie partiellement le HASH, puis recalcule l'alerte (le seuil ayant pu changer).
- 4 **Suppression** — `delete_product()` supprime, via une **pipeline** Redis atomique, les trois clés du produit ainsi que son entrée dans le ZSET d'alertes.

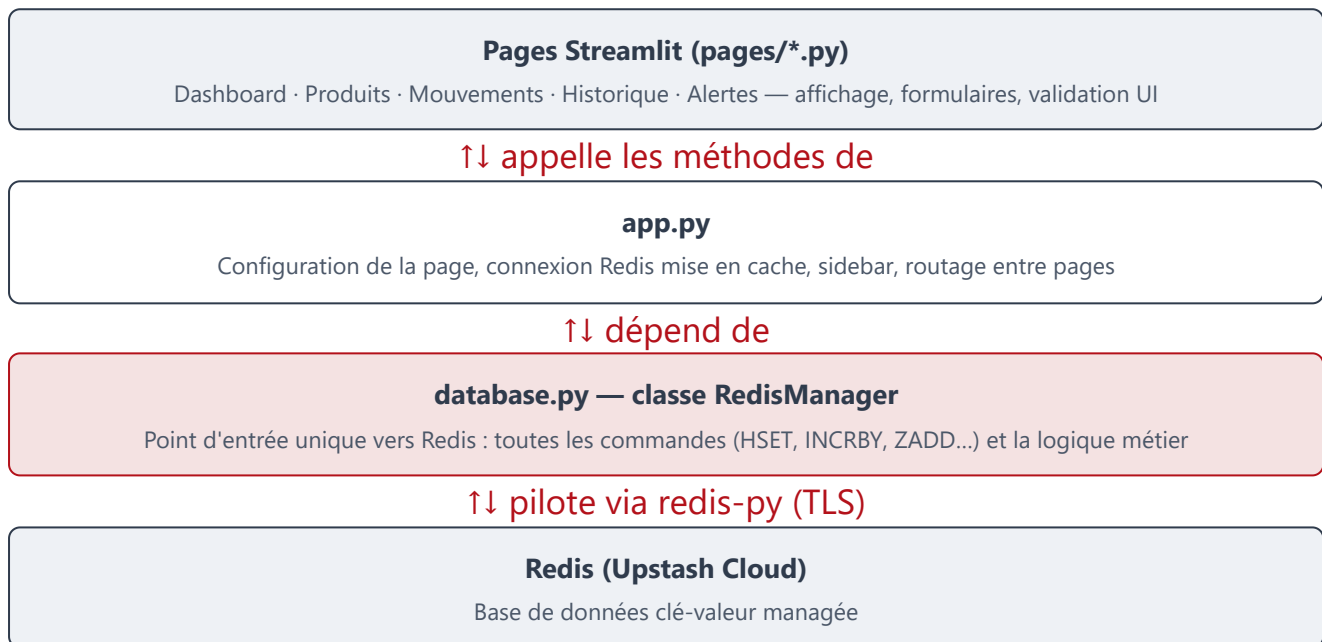
Intégrité des données. Toute opération qui modifie le stock déclenche systématiquement un recalcul de la position du produit dans `alertes:stock_bas` (`update_alerts()`), garantissant que le ZSET

d'alertes reste toujours cohérent avec l'état réel du stock, sans tâche de synchronisation séparée.

Architecture logicielle

7.1 — Architecture en couches

L'application respecte une séparation stricte entre l'interface, le routage et l'accès aux données, imposée par le cahier des charges : **aucune commande Redis n'est exécutée en dehors du fichier `database.py`**.



7.2 — Organisation des fichiers

```

mon-projet-nosql/
├─ app.py                # Point d'entrée : connexion, sidebar, routage
├─ database.py           # Classe RedisManager – toute la logique Redis
├─ requirements.txt      # streamlit, redis, python-dotenv
├─ .env.example          # Modèle de variables d'environnement
├─
├─ pages/
│   ├─ dashboard.py      # KPI, alertes, top 3 stocks faibles
│   ├─ produits.py       # CRUD produits (liste, ajout, modif, suppr)
│   ├─ mouvements.py     # Entrées / sorties de stock
│   ├─ historique.py     # Journal des mouvements par produit
│   └─ alertes.py        # Liste des produits en rupture
├─

```

```
└─ utils/  
   └─ helpers.py
```

```
# Formatage, validation, badges réutilisables
```

7.3 — Flux d'une requête utilisateur

- 1 L'utilisateur ouvre l'application ; `app.py` établit (ou réutilise, via `st.cache_resource`) une connexion unique à Redis pour toute la session serveur.
- 2 La sidebar affiche la navigation et interroge `manager.get_alerts()` pour un résumé rapide des alertes, visible depuis n'importe quelle page.
- 3 Selon la page choisie, `route_page()` importe dynamiquement le module correspondant dans `pages/` et lui délègue l'affichage.
- 4 La page appelle exclusivement les méthodes publiques de `RedisManager` (jamais une commande Redis brute) pour lire ou écrire des données.
- 5 Le résultat est mis en forme via les utilitaires de `utils/helpers.py` (devise, dates, badges) puis rendu par les composants Streamlit.

7.4 — Choix d'implémentation notables

- **Connexion mise en cache** (`st.cache_resource`) : une seule connexion Redis est ouverte par session serveur, évitant une reconnexion TLS à chaque interaction utilisateur.
- **SCAN plutôt que KEYS** : `get_all_products()` utilise `scan_iter()`, une itération non bloquante, pour rester performant même avec un grand volume de produits en production.
- **Pipelines Redis** : les opérations multi-clés (suppression d'un produit, ajout d'un mouvement + troncature de l'historique, incrément de compteur + pose du TTL) sont regroupées dans des `pipeline()` pour minimiser les allers-retours réseau.
- **Exceptions métier dédiées** : `ProduitInexistantError`, `ProduitDejaExistantError`, `StockInsuffisantError` permettent à la couche interface d'afficher des messages d'erreur explicites plutôt que des erreurs techniques génériques.

Fonctionnement détaillé de l'application

8.1 — 🏠 Dashboard

Page d'accueil offrant une vue synthétique de l'activité : quatre indicateurs clés (produits référencés, produits en alerte, entrées et sorties des dernières 24h), suivis du top 3 des stocks les plus faibles, d'un aperçu des cinq alertes les plus critiques, et enfin d'un tableau complet de tous les produits avec leur état (OK / Alerte / Rupture).

8.2 — 📁 Produits

Page organisée en quatre onglets :

- **Liste & recherche** — tableau de tous les produits, filtrable par SKU ou nom en temps réel.
- **Ajouter** — formulaire de création avec validation (SKU, nom et fournisseur obligatoires, prix strictement positif) et stock initial optionnel.
- **Modifier** — sélection d'un produit existant, formulaire pré-rempli, mise à jour partielle des champs modifiés.
- **Supprimer** — sélection, avertissement explicite sur l'irréversibilité, case de confirmation obligatoire avant activation du bouton de suppression.

8.3 — 🔄 Mouvements

Deux onglets symétriques, **Entrée** et **Sortie** de stock. Chaque formulaire propose la liste des produits (avec leur stock actuel affiché dans le libellé), une quantité et un commentaire libre. Une sortie supérieure au stock disponible est bloquée avec un message d'erreur explicite (« Stock insuffisant »), garantissant qu'aucun stock n'est jamais négatif.

8.4 — 📄 Historique

Après sélection d'un produit, la page affiche son état courant (stock, seuil, statut), puis la liste chronologique de ses mouvements récents sous forme de journal coloré (vert pour les entrées, rouge pour les sorties), doublée d'une vue tabulaire dépliable pour l'export ou la copie.

8.5 — 🚨 Alertes

Liste complète des produits sous seuil, déjà triée par urgence grâce au Sorted Set Redis sous-jacent. Chaque ligne affiche un niveau de sévérité calculé côté interface : *rupture de stock* (stock ≤ 0), *très critique* (stock ≤ 50 % du seuil), ou *stock bas* (en dessous du seuil).

Page	Structures Redis mobilisées	Méthodes RedisManager principales
Dashboard	HASH, STRING, ZSET, STRING+TTL	<code>get_statistics()</code> , <code>get_alerts()</code> , <code>get_all_products()</code>
Produits	HASH, STRING	<code>add_product()</code> , <code>update_product()</code> , <code>delete_product()</code> , <code>get_all_products()</code>
Mouvements	STRING, LIST, ZSET, STRING+TTL	<code>add_stock()</code> , <code>remove_stock()</code>
Historique	LIST	<code>get_history()</code>
Alertes	ZSET	<code>get_alerts()</code>

Règles métier et gestion des erreurs

9.1 — Règles de gestion

- Un SKU doit être unique : la création échoue si le produit existe déjà.
- Le stock d'un produit ne peut **jamais devenir négatif** : toute sortie est vérifiée contre le stock disponible avant application.
- Les quantités d'entrée et de sortie doivent être strictement positives.
- Une alerte est déclenchée dès que `stock < seuil_alerte` (strictement) ; elle est levée automatiquement dès que le stock repasse au-dessus.
- L'historique d'un produit est borné à 50 mouvements ; les plus anciens sont automatiquement purgés (`LTRIM`).
- La suppression d'un produit est irréversible et purge toutes ses données associées (stock, historique, alerte).

9.2 — Hiérarchie des exceptions métier

Exception	Déclenchée quand...	Message affiché
<code>ProduitDejaExistantError</code>	Création d'un produit avec un SKU déjà utilisé	« Le produit '<sku>' existe déjà. »
<code>ProduitInexistantError</code>	Opération (modification, suppression, mouvement) sur un SKU inconnu	« Le produit '<sku>' n'existe pas. »
<code>StockInsuffisantError</code>	Sortie de stock supérieure à la quantité disponible	« Stock insuffisant pour '<sku>' : disponible X, demandé Y. »
<code>ValueError</code>	Quantité d'entrée/sortie nulle ou négative, <code>REDIS_URL</code> absent	Message contextuel

Chaque page Streamlit capture ces exceptions spécifiquement pour afficher un message d'erreur métier clair (`st.error()`), tout en conservant une capture générique de secours pour les erreurs techniques imprévues (coupure réseau, timeout Upstash), sans jamais faire planter l'application.

Sécurité et configuration

10.1 — Gestion des secrets

L'URL de connexion à Redis (incluant les identifiants d'authentification) n'est jamais écrite en dur dans le code source. Elle est chargée depuis une variable d'environnement `REDIS_URL`, elle-même définie :

- en local, via un fichier `.env` (chargé par `python-dotenv`), explicitement exclu du dépôt Git via `.gitignore` ;
- en production, via les *Secrets* du tableau de bord Streamlit Community Cloud.

Un fichier `.env.example` documente le format attendu sans exposer de valeur réelle, facilitant l'onboarding sans risque de fuite d'identifiants.

10.2 — Chiffrement du transport

La connexion à Upstash utilise le schéma `rediss://` (Redis over TLS) : l'ensemble du trafic entre l'application et la base de données est chiffré en transit. Des timeouts explicites de connexion et de lecture (5 secondes) évitent qu'une indisponibilité réseau ne bloque indéfiniment l'application.

10.3 — Robustesse applicative

L'échec de connexion à Redis (URL manquante, identifiants invalides, base injoignable) est intercepté dès le démarrage de l'application (`init_app()`) et affiché sous forme de message d'erreur explicite à l'utilisateur, plutôt que de provoquer un plantage silencieux de l'interface.

Limite assumée. L'application ne met en œuvre **aucune authentification utilisateur** : il s'agit d'un choix délibéré, cohérent avec le périmètre pédagogique du module (démonstration de la modélisation NoSQL), mais qui devrait impérativement être adressé avant tout usage en production réelle (cf. chapitre 13).

Installation et déploiement

11.1 — Exécution en local

- 1 Cloner le dépôt GitHub et se placer dans le dossier du projet.
- 2 Créer un environnement virtuel Python (`python3 -m venv venv`) puis l'activer.
- 3 Installer les dépendances : `pip install -r requirements.txt` .
- 4 Copier `.env.example` en `.env` et y renseigner sa propre `REDIS_URL` Upstash.
- 5 Lancer l'application : `streamlit run app.py` — accessible sur `http://localhost:8501` .

11.2 — Provisionnement de la base Redis (Upstash)

- 1 Créer un compte gratuit sur `console.upstash.com` (aucune carte bancaire requise).
- 2 Créer une base *Regional* dans une région proche des utilisateurs.
- 3 Récupérer l'URL de connexion au format TLS (`rediss://...`) depuis l'onglet « Redis Connect ».
- 4 Renseigner cette URL dans `.env` (local) ou dans les Secrets Streamlit Cloud (production).

Le plan gratuit Upstash autorise jusqu'à 10 000 commandes/jour, largement suffisant pour ce projet académique.

11.3 — Déploiement sur Streamlit Community Cloud

- 1 Pousser le projet sur un dépôt GitHub **public** (condition imposée par le barème du module).
- 2 Sur `share.streamlit.io` , créer une nouvelle application en pointant vers le dépôt et le fichier `app.py` .
- 3 Renseigner `REDIS_URL` dans les *Advanced settings* > *Secrets* (jamais commitée dans le dépôt).
- 4 Déployer : l'application devient accessible via une URL publique Streamlit.

Application déployée : `https://didier-gloire-aude.streamlit.app/`

Dépôt GitHub : `https://github.com/juniorkema364/streamlit_project`

Stack technique

Composant	Technologie	Rôle dans le projet
Langage	Python 3.10+	Langage unique du projet, front et back
Interface web	Streamlit 1.59	Rendu de l'interface, formulaires, navigation, mise en cache de ressources
Base de données	Redis 5.0 (client <code>redis-py</code>)	Stockage clé-valeur de l'ensemble des données métier
Hébergement base	Upstash (Redis managé, cloud)	Base Redis accessible en TLS, plan gratuit
Gestion des secrets	python-dotenv	Chargement de <code>REDIS_URL</code> depuis <code>.env</code> en local
Hébergement application	Streamlit Community Cloud	Déploiement public de l'application
Versionnement	Git / GitHub	Suivi de version et dépôt public du code source

Limites et perspectives d'évolution

13.1 — Limites actuelles

- **Absence d'authentification** : tout utilisateur ayant l'URL de l'application peut consulter et modifier l'intégralité du stock.
- **Acteur unique** : aucune distinction de rôle (magasinier, responsable, administrateur) ni de traçabilité de « qui » a effectué un mouvement.
- **Historique borné** : au-delà de 50 mouvements, les plus anciens sont perdus — un usage d'audit long terme nécessiterait un export périodique vers un stockage froid.
- **Absence de gestion multi-magasin** : le modèle actuel suppose un référentiel de stock unique, sans notion de dépôt ou de point de vente.
- **Pas de notifications proactives** : les alertes sont consultées à la demande, sans envoi automatique (email, SMS) lors du franchissement d'un seuil.

13.2 — Perspectives d'évolution

- Ajout d'une authentification utilisateur (via `streamlit-authenticator` ou un fournisseur OAuth) et d'une journalisation nominative des mouvements.
- Archivage périodique de l'historique complet vers une base secondaire (par exemple une base documentaire) pour un audit long terme, tout en conservant Redis pour les accès chauds.
- Extension du modèle de clés pour supporter plusieurs entrepôts (`stock:<depot>:<sku>`).
- Envoi de notifications automatiques (email/webhook) lors du déclenchement d'une alerte critique, en s'appuyant sur les capacités Pub/Sub de Redis.
- Ajout de graphiques d'évolution du stock dans le temps (nécessiterait une structure de série temporelle complémentaire, par exemple via Redis TimeSeries).

Conclusion

Le projet **GestiStock** démontre qu'une base de données clé-valeur comme Redis, loin d'être une simple alternative « plus rapide mais plus limitée » à un SGBD relationnel, propose un véritable vocabulaire de structures de données (HASH, STRING, LIST, ZSET, TTL) qui, correctement choisies, permettent de résoudre élégamment et efficacement un cas d'usage métier réel de gestion de stock : atomicité des mouvements, tri automatique des priorités d'alerte, historique borné sans purge manuelle, statistiques auto-expirantes.

L'architecture retenue — séparation stricte entre interface Streamlit, routage applicatif et couche d'accès Redis centralisée dans `RedisManager` — garantit par ailleurs que toute évolution future du modèle de données (nouvelle structure, nouvelle règle métier) peut être apportée sans jamais toucher à la logique d'affichage, ce qui constitue une base saine pour les évolutions envisagées au chapitre précédent.

Ce travail a permis au Groupe 3 de consolider une compréhension pratique des principes de modélisation NoSQL orientée clé-valeur, en les confrontant directement aux contraintes d'une application déployée et utilisable.



GestiStock — Rapport de projet NoSQL

Présenté par le Groupe 3 : KEMA Didier, Mouanga Gloire, MATSIONA Aude

Présenté par M. OBANDA Jefferson — ECES, 3^e année Data Science, 2025/2026